

Week06 | 课时 1 | 为什么脚本跑通不等于数据产品可运营

本课导航

跑通脚本只是开始，能运营资产才是系统能力	1
这节课解决什么问题	1
参考学习时间	1
学完这一讲，你应该能做到什么	2
本课产出	2
先看一张总图	2
1. 脚本跑通为什么仍然不可运营	3
2. 任务流和资产流的区别	3
3. 从任务流升级到资产流	4
任务日志 / Pipeline Run / Asset State / Run Evidence	4
4. 在 OmniSupport Copilot 里，哪些东西应该被看成资产？	5
5. 为什么 AI 数据工程更需要资产状态	5
6. Week06 的最小升级路径	5
5 分钟练习：把脚本结果改写成资产判断	6
常见误解	6
自检清单	7
课后最小行动	7
延伸阅读	7
Footnotes	7

跑通脚本只是开始，能运营资产才是系统能力

这一讲先立住 Week06 的问题意识：

AI 数据工程的失败，很多时候不是脚本没跑，而是团队不知道哪些数据资产处于什么状态。

这节课解决什么问题

Week03 已经让我们有了 ingest、checkpoint、replay、backfill 的脚本思维。问题是：

如果链路由多个脚本、表、指标和未来索引组成，团队靠什么判断今天的数据产品是否可消费？

靠脚本日志不够，靠口头顺序不够，靠“我刚才跑过了”更不够。

周一早上，运营同学发现 P1 工单数突然少了一半。工程同学看日志，`ticket_ingest.py` 昨晚是 success，Week05 的 dbt 也跑过。但没人能马上回答：昨天的 source manifest 是哪一版，2026-04-17 分区是不是完整，`support_kpi_mart` 有没有接住，run evidence 有没有生成。最后的问题不是脚本没跑，而是团队没有一个共同判断资产状态的地方。

Week06 要解决的就是：团队如何共同相信一个数据资产。

参考学习时间

45–55 分钟

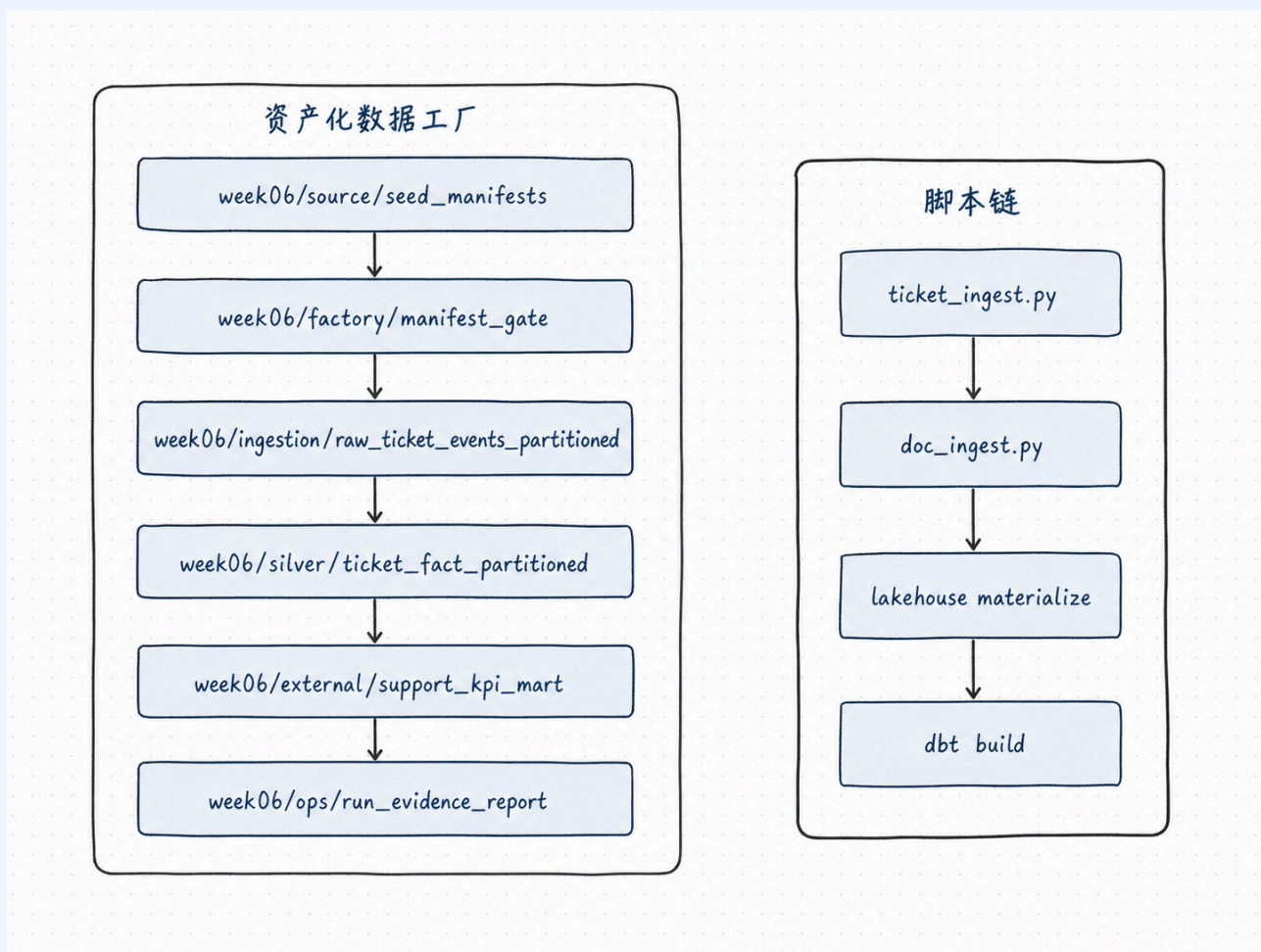
学完这一讲，你应该能做到什么

1. 区分“任务流”和“资产流”。
2. 解释为什么 Week03 的 replay/backfill 需要升级成 asset-level recovery。
3. 说清 AI 数据链路为什么必须有资产状态、分区状态和运行证据。
4. 识别脚本堆叠带来的交接、回填和下游阻断风险。
5. 说明 Week06 为什么不是 Dagster UI 教程。

本课产出

- docs/blueprints/week06/week06-data-factory-blueprint.md
- docs/blueprints/week06/week06-asset-graph.md
- 一段把“昨天我跑过 ticket ingest 了”改写成资产判断的表达

先看一张总图



这张图的差别不在“换了工具”，而在：

- 左边关心脚本是否跑过；
- 右边关心资产是否可消费；
- 左边依赖人工记忆；
- 右边记录依赖、分区、检查和证据。

1. 脚本跑通为什么仍然不可运营

脚本 success 只能说明“某段代码在某台机器、某个时间点没有抛异常”。它不能自动说明输入是否完整、分区是否正确、下游是否同步、质量检查是否通过、证据是否能被别人复核。

⚠ 事故小剧场

A 同学在本机跑完 `ticket_ingest.py`，日志里出现 success。B 同学第二天接手时只知道“昨天跑过”，但不知道 `week06/silver/ticket_fact_partitioned` 哪个分区完成、Week05 的 `support_kpi_mart` 是否同步、run evidence 是否生成。下游如果继续做 Week08 index，很可能把一个缺证据的状态当成正式 baseline。

脚本式 pipeline 最容易留下四类盲区：

盲区	表现	后果
顺序盲区	只知道先跑哪个脚本	换人后无法复现
状态盲区	不知道哪个分区成功	补数靠猜
质量盲区	跑完没有 checks	坏数据进入下游
证据盲区	没有 run evidence	评测、追踪、治理断链

团队真正会问的不是“脚本是不是跑过”，而是：

1. 今天哪些 asset 可以被下游消费？
2. 哪个 partition 是最新可信状态？
3. 上游 manifest 是哪一版？
4. 输入和输出行数是否对得上？
5. 有没有重复写入或漏数？
6. 如果失败，应该 retry、replay 还是 backfill？
7. Week04 / Week05 依赖没启用时，是 blocked、skipped 还是 not_available？
8. 证据 JSON 在哪里，schema 是否校验过？
9. 谁能批准下游 unblock？
10. 另一个人能不能照 runbook 复现恢复路径？

2. 任务流和资产流的区别

任务流关注：

- 运行了哪个脚本；
- 脚本 exit code 是什么；
- log 里有没有报错。

资产流关注：

- 哪个 asset 被 materialize；
- 上游依赖是否满足；
- 哪个 partition 成功或失败；
- 哪些 checks 通过；
- 证据 report 在哪里；
- 下游是否应该放行。

维度	任务流	资产流
关注对象	运行了哪个脚本	哪个数据资产处于什么状态
失败边界	某个命令失败	某个 asset / partition 失败
恢复动作	重跑脚本	retry / replay / backfill / hold downstream
证据	日志	materialization metadata + checks + evidence report
适合阶段	个人开发、早期 demo	团队交接、上线、回归、治理

不是说任务流没用。任务流是把事情跑起来的起点；Week06 要做的是在任务流之上建立资产状态层，让团队知道“跑完以后系统现在能相信什么”。

! 核心判断

任务流回答“我跑了什么”；资产流回答“系统现在能相信什么”。

3. 从任务流升级到资产流



这张图要表达的不是工具链更复杂，而是判断口径更完整。只有走到 evidence 和 downstream decision，另一个人才能复核、放行或追责。停在 script success，团队仍然只能相信作者本人的说法。

任务日志 / Pipeline Run / Asset State / Run Evidence

对象	小白版类比	它保护什么	它不能替代什么
脚本日志	厨师的操作记录	说明某段代码运行时发生了什么	不能证明下游可消费
pipeline run	一次工单流程结束	说明一组任务被调度和执行过	不能表达具体 asset 的业务状态
asset state	某个货架上的商品状态	说明某个资产、某个分区现在是否完成、失败或跳过	不能替代结构化证据归档
run evidence	验收单	记录运行事实、reason code、report path、downstream decision	不能替代代码测试和人工 runbook

工程上，这四层都需要，但不能互相冒充。日志适合排查细节，pipeline run 适合看调度过程，asset state 适合判断数据产品边界，run evidence 适合交付、复核、评测和治理。

4. 在 OmniSupport Copilot 里，哪些东西应该被看成资产？

项目对象	为什么是资产	Week06 关注什么
<code>week06/source/seed_manifests</code>	输入声明决定这次运行读了什么	manifest 数量、manifest id、是否有结构化 ticket manifest
<code>week06/factory/manifest_gate</code>	gate 决定 source 是否允许进入本轮数据工厂	是否 accepted、reason code、batch id
<code>week06/ingestion/raw_ticket_events_partitioned</code>	工单原始事实是下游事实层的输入	哪个 daily partition 完成，输入行数是否合理
<code>week06/silver/ticket_fact_partitioned</code>	结构化事实层是后续指标、检索和评测的核心消费对象	是否重复、漏数、空字段、输出行数是否可解释
<code>week06/external/support_kpi_mart</code>	Week05 指标层会影响后续 Agent 和 BI 消费	作为 optional observation；缺失时写 <code>not_available</code> ，不伪造 passed
<code>week06/ops/run_evidence_report</code>	运行证据把资产状态、检查和下游决策串起来	证据 JSON 是否符合 <code>contracts/run_evidence/week06_run_evidence.schema.js</code>

这张表的重点是：Week06 不要求你把所有函数都变成 asset。它先把会影响下游消费、恢复和交接的对象变成 asset，并且给每个对象留下可复核的状态。

5. 为什么 AI 数据工程更需要资产状态

传统 BI 看板错了，通常是人发现后再查 SQL、修口径、重刷报表。AI 数据链路更危险：错误会继续进入解析、索引、Agent、评测和发布，扩散更快，也更难定位。

具体有四个原因：

1. RAG 索引会固化当时的数据状态；如果错误数据被索引，后面查询会持续命中旧错误。
2. Agent 工具可能基于错误指标做动作；工具看起来答得很顺，但基础数据已经错了。
3. 评测需要绑定输入版本；否则一次评测失败无法判断是模型变了、数据变了，还是运行状态变了。
4. 治理需要知道谁、何时、基于什么数据放行；没有 run evidence，事故复盘只能靠聊天记录和记忆。

6. Week06 的最小升级路径

Week06 不是重写 Week03–Week05，而是做薄包装：

```
existing ingest scripts
-> Dagster assets
-> daily partitions
-> asset checks
-> run evidence
-> data factory runbook
```


i 讲师可选演示

可以在课堂上打开项目 runbook, 展示 `week06/source/seed_manifests` 到 `week06/ops/run_evidence_report` 的 asset key 列表。演示重点不是点 UI, 而是让学员看到“每个 asset 都有名字、依赖、状态和证据落点”。¹

5 分钟练习：把脚本结果改写成资产判断

给出一句脚本式表达：

昨天我跑过 ticket ingest 了。

请改写成资产式表达：

- 哪个 asset?
- 哪个 partition?
- 输入 manifest 是谁?
- 哪些 checks 通过?
- evidence report 在哪里?
- 下游决策是什么?

参考表达：

`week06/silver/ticket_fact_partitioned` 的 `2026-04-17` 分区已基于某个 structured manifest 完成 dry-run materialization, 核心 checks 通过, run evidence 写入 `reports/week06/run_evidence/`, 当前 downstream decision 是 `dry_run_only`, 不能直接宣称生产放行。

常见误解

误解	纠偏
Dagster 的价值是 UI 好看	UI 只是表象, 核心是 asset state model ²
脚本能重跑就等于可恢复	没有分区边界和幂等证据, 重跑仍然危险
我有日志, 所以我有证据	日志没有统一 schema, 难以被下游、评测和治理消费
我把所有函数都做成 asset, 越细越好	asset 粒度要围绕数据产品、恢复边界和交接边界, 而不是函数数量
Week06 就是把 Week03 再包一遍	Week06 的价值是资产状态、检查、证据和交接, 不是代码包装
有 Dagster UI 就有数据工厂	没有 asset key、checks、evidence、runbook, UI 只是展示层

¹当前项目 runbook 中列出的 Week06 asset group 包括 `week06/source/seed_manifests`、`week06/factory/manifest_gate`、`week06/ingestion/raw_ticket_events_partitioned`、`week06/silver/ticket_fact_partitioned`、`week06/external/support_kpi_mart` 和 `week06/ops/run_evidence_report`。

²Dagster 的 software-defined assets 强调用代码定义数据资产和依赖关系；本课借用的是这个资产状态模型，而不是把 Dagster UI 当成学习目标。

资产图越复杂越专业

Student Core 先要少量关键资产和清晰命名

runbook 是最后补文档

runbook 是系统设计的一部分

自检清单

- ☐ 我能解释任务流和资产流的差别
- ☐ 我能说清为什么“跑过”不等于“可消费”
- ☐ 我能指出 Week03 replay/backfill 的资产化升级方向
- ☐ 我知道 Week06 不能抢跑 Week07 / Week08 / Week14
- ☐ 我能写出一条最小资产化升级路径
- ☐ 我能说明失败后应该补哪个 partition, 而不是只说“重跑一下”
- ☐ 我能把运行证据交给另一个人复核
- ☐ 我能把一句脚本式表达改写成 asset + partition + checks + evidence + decision

课后最小行动

在项目笔记中写下：

```
## Week06 operational asset risks
```

- 哪些脚本目前只靠人工顺序运行
- 哪些数据资产缺少分区状态
- 哪些结果缺少 checks
- 哪些 report 可以作为 run evidence 起点
- 哪些下游决策现在还只能靠口头确认

延伸阅读

主题	推荐资料	为什么读
Software-defined assets	Dagster Docs: Assets	理解 asset 不是普通任务节点, 而是数据产品边界
Asset materialization meta-data	Dagster Docs: Asset materializations and metadata	理解 materialization 为什么要携带行数、路径、状态等证据
Asset observations	Dagster Docs: Asset observations	理解观察 external asset 时, 为什么不等于修改了它 ³
OpenLineage object model	OpenLineage Docs: Object model	理解 run、job、dataset 为什么适合后续治理追溯 ⁴
Week06 项目 runbook	runbooks/week06-data-factory.md	对齐真实 asset key、默认分区、checks、evidence 和恢复决策

Footnotes

³Dagster asset observation 用来记录外部资产的观察事实, 不代表本次运行修改了这个资产; 这正好对应 Week06 对 Week04 / Week05 optional dependency 的诚实记录方式。

⁴OpenLineage 的 run / job / dataset 模型适合描述“哪次运行、哪个任务、读写了哪些数据集”; Week06 先预埋这种追溯观念, 完整治理平台留到后续周展开。